

Cardoso Hub HTTPS — Configuration Guide

Caddy + Tailscale, end to end.

This guide walks through configuring the Cardoso Hub to serve its UI and API over HTTPS via Caddy, using a Let's Encrypt certificate issued through Tailscale's built-in `tailscale cert` workflow.

By the end of this guide you will have:

- A Hub answering on `https://<machine>.<tailnet>.ts.net` with a real, publicly-trusted TLS certificate.
- The Cardoso Node service bound to `127.0.0.1` only — no LAN exposure.
- Caddy running as a Windows service (`CardosoCaddy`) that survives reboots.
- Automatic certificate renewal every ~60 days, no operator action required.

The whole setup takes about **10 minutes** of hands-on work plus ~1 minute of script runtime.

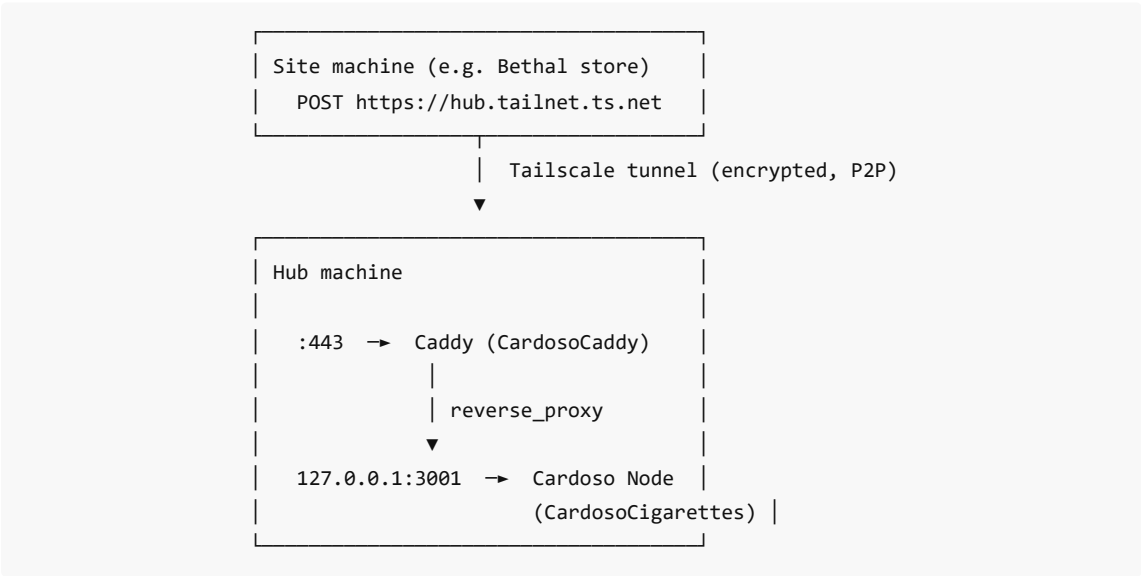
1. Why this design

Most "put HTTPS in front of an internal app" guides assume you have a public DNS name and an open port 80/443 for Let's Encrypt's HTTP-01 challenge. The Cardoso Hub has neither — it lives entirely on a Tailscale tailnet, with no public DNS and no inbound ports open from the internet. So the conventional Caddy auto-HTTPS doesn't work.

What we use instead:

- **Tailscale's `tailscale cert` command.** Tailscale brokers a Let's Encrypt certificate over the DNS-01 challenge using its control plane. No public DNS record, no open ports — the cert just appears on disk for any `*.<tailnet>.ts.net` name your tailnet owns.
- **Caddy in "use existing cert" mode.** Caddy normally manages its own certs; we instead point it at the `.cert` / `.key` files Tailscale wrote, and Caddy serves them. Caddy still does TLS termination, HTTP/2, and reverse-proxy duties for us.
- **Cardoso Node bound to localhost.** Once Caddy is verified working, we switch the Node service from `0.0.0.0:3001` to `127.0.0.1:3001`. The LAN can no longer reach Node directly; only Caddy on the same machine can.

The end-state architecture:



The benefits:

- **No public attack surface.** Nothing of the Hub is reachable from the open internet — only from devices on the tailnet.
 - **Real, trusted certificates.** The browser's lock icon is green, no certificate warnings, no manually-installed CA roots on every site machine.
 - **Automatic renewal.** Tailscale auto-renews the cert about 30 days before expiry as long as the machine stays on the tailnet.
 - **Defense in depth.** Even if someone got onto the LAN (not the tailnet), they couldn't reach the Hub directly because Node is bound to loopback.
-

2. Prerequisites

Before starting, confirm all of these:

2.1 Cardoso Customer App is installed

The standard `CardosoSetup-*.exe` installer should already be at:

```
C:\Cardoso Customer App\
```

This matters because the Caddy installer reuses **NSSM** (`nssm.exe`) from that directory to register Caddy as a Windows service — saving us from shipping a second copy of NSSM.

2.2 Tailscale is installed and connected

Run on the Hub:

```
tailscale status
```

You should see your tailnet listed and the Hub machine's name. If `tailscale` isn't recognised, install it from <https://tailscale.com/download/windows> and run `tailscale login`.

2.3 HTTPS for tailnet machines is enabled

This is the **single most-missed step**. Open the Tailscale admin panel:

<https://login.tailscale.com/admin/dns>

Scroll to **HTTPS Certificates** and toggle it **ON**. Without this, every `tailscale cert` invocation will fail with a confusing error about being unable to issue certificates.

2.4 You know the Hub's tailnet hostname

Run on the Hub:

```
tailscale status
```

The first column for the local machine is the hostname you'll use, in the form:

```
<machine>.<tailnet>.ts.net
```

For example: `cardoso-hub.tail-9a8b7.ts.net` . Write this down — you need it for the next section.

2.5 An elevated PowerShell

You'll need to run scripts and modify Windows services, so launch PowerShell **as Administrator** for the install steps.

3. Installing Caddy

The Caddy install is fully scripted. From an elevated PowerShell on the Hub:

```
cd "C:\Cardoso Customer App\scripts"  
.\install-hub-caddy.ps1 -Hostname "<machine>.<tailnet>.ts.net"
```

That's it. The script is **idempotent** — you can re-run it any time and it will pick up where it left off without breaking anything.

What the script does

It runs through 8 numbered steps, all visible in the console:

1. **Prerequisite checks.** Verifies `nssm.exe` exists at the expected path and that `tailscale` is on PATH and connected. Aborts with a clear error message if anything's missing.
2. **Caddy download.** Pulls the latest stable Caddy binary into `C:\Caddy\caddy.exe` . Skipped if the binary already exists. Use `-SkipDownload` to force-reuse an existing binary.
3. **Cert issuance.** Runs `tailscale cert <hostname>` , which writes `C:\Caddy\<hostname>.crt` and `C:\Caddy\<hostname>.key` . This is the step that requires HTTPS Certificates to be enabled in the admin panel — if it's off, you get the error here.
4. **Caddyfile generation.** Writes `C:\Caddy\Caddyfile` with a minimal reverse-proxy config (see Appendix A for the full content).
5. **Caddyfile validation.** Runs `caddy validate` to confirm the file parses cleanly. If you ever hand-edit the Caddyfile, this is the same command to use.
6. **Service registration.** Uses NSSM to register `CardosoCaddy` as a Windows service: auto-start, restart-on-failure, logs going to `C:\Caddy\logs\service*.log` .
7. **Service start.** Starts `CardosoCaddy` . Waits a couple of seconds for it to come up.
8. **Smoke test.** Hits `https://<hostname>/api/health` and expects a 200. If Node isn't running yet, this will fail — that's OK, just confirms that Caddy itself is up.

Total runtime: about a minute. If any step fails, the script aborts with the failing step number and a hint about how to fix it.

What you should see at the end

```
Done. Caddy is serving the Hub on https://<machine>.<tailnet>.ts.net  
Smoke test: 200 OK
```

If the smoke test failed because Node wasn't running, that's expected at this stage — keep going.

4. Locking the Node service to localhost

Caddy is now reverse-proxying HTTPS traffic to Node, but Node is still listening on `0.0.0.0:3001` — meaning the LAN can still talk to it directly, bypassing Caddy entirely. We close that path now.

4.1 Edit the Node .env

Open:

```
C:\Cardoso Customer App\.env
```

Add the line:

```
BIND_ADDRESS=127.0.0.1
```

Save the file.

4.2 Restart the Node service

```
& "C:\Cardoso Customer App\nssm\nssm.exe" restart CardosoCigarettes
```

4.3 Verify the binding

```
netstat -an | Select-String ":3001"
```

You should see a single line like:

```
TCP        127.0.0.1:3001    0.0.0.0:0        LISTENING
```

You should **not** see `0.0.0.0:3001`. If you do, the env var didn't take — double-check the `.env` for typos and that you saved before restarting.

Why this is "defense in depth"

Even if Caddy were misconfigured tomorrow and started forwarding to the wrong host, an attacker on the LAN still couldn't reach the Cardoso Node directly, because Node is no longer accepting non-loopback connections. This matters because the audit trail, customer balances, and BAT reconciliation data live in Node — Caddy is just the front door.

5. Re-enabling HTTPS-only security headers

The Cardoso server has `helmet` middleware that applies HTTPS-only security headers (HSTS, `upgrade-insecure-requests`, `Cross-Origin-Opener-Policy`, etc.). These were disabled by default for HTTP-on-LAN compatibility. Now that HTTPS is in front, we turn them back on.

5.1 Edit .env again

Add:

```
TLS_FRONTING=true
```

So the file now contains both lines:

```
BIND_ADDRESS=127.0.0.1
TLS_FRONTING=true
```

5.2 Restart the Node service

```
& "C:\Cardoso Customer App\nssm\nssm.exe" restart CardosoCigarettes
```

5.3 What to expect

The boot log (visible in `C:\Cardoso Customer App\logs\service.log`) should now show:

```
[system.boot] Server started on port 3001 (... bind=127.0.0.1, tls_fronting=true)
```

Browse the Hub. In the browser DevTools (F12) → **Network** tab, click any request, look at **Response Headers**. You should see:

- Strict-Transport-Security: max-age=15552000
- Content-Security-Policy: ... upgrade-insecure-requests ...
- Cross-Origin-Opener-Policy: same-origin
- Origin-Agent-Cluster: ?1

Click around the app — Customer Search, BAT Reconciliation, Reports. If anything visibly breaks (charts not rendering, BAT preview iframe blank, fonts missing) it means the CSP needs tuning. Set `TLS_FRONTING=false` and restart while you fix the CSP directives in [server.js](#) before re-enabling.

6. Updating the sites

Each site machine has a `HUB_URL` environment variable that tells it where to push reconciliation data. With HTTPS now in front of the Hub, every site needs that URL updated.

6.1 Find the current URL

On a site machine, open:

```
C:\Cardoso Customer App\.env
```

Look for `HUB_URL` . It will currently be something like:

```
HUB_URL=http://cardoso-hub.tail-9a8b7.ts.net:3001
```

6.2 Change it

Update to:

```
HUB_URL=https://cardoso-hub.tail-9a8b7.ts.net
```

Two changes: `http` → `https` , and the `:3001` is removed (HTTPS uses port 443 implicitly).

6.3 Restart the site service

```
& "C:\Cardoso Customer App\nssm\nssm.exe" restart CardosoCigarettes
```

6.4 Repeat for every site

Do this on every site machine. The next site EXE installer release ships the new default, but existing installs need to be updated by hand until they upgrade.

Tip: if you have a lot of sites, the **Settings** → **TLS** tab on the Hub now surfaces the current TLS posture so you can verify the Hub side is correct without RDP'ing in.

7. Verification checklist

Run through this on the Hub after install. Every check should pass.

7.1 Caddy is running

```
Get-Service CardosoCaddy
```

Status: Running

7.2 Caddy is bound to 443

```
netstat -an | Select-String ":443"
```

Should show `LISTENING` on `0.0.0.0:443`.

7.3 Cert files exist on disk

```
Test-Path "C:\Caddy\<hostname>.crt" # True
Test-Path "C:\Caddy\<hostname>.key" # True
```

7.4 The Hub answers over HTTPS

```
Invoke-WebRequest -Uri "https://<hostname>/api/health" -UseBasicParsing
```

StatusCode: 200

7.5 Node is bound to localhost only

```
netstat -an | Select-String "127.0.0.1:3001" # LISTENING
netstat -an | Select-String "0.0.0.0:3001" # NOTHING
```

7.6 The browser shows a valid lock icon

```
Start-Process "https://<hostname>"
```

The browser opens, shows the green lock icon, **no certificate warnings**, the Cardoso login screen renders normally.

7.7 The TLS settings tab confirms posture

In the Hub UI, log in as admin → **Settings** → **TLS**. You should see:

- Posture badge: **TLS fronted** (green)
- Caddy: installed at `C:\Caddy`
- Caddyfile hostname matches yours
- Cert: valid, days-until-expiry > 60
- Service: `CardosoCaddy` — running

If any of these are off, the panel tells you which one.

8. Day-to-day operations

8.1 Service control

```
# Status
Get-Service CardosoCaddy

# Restart after editing the Caddyfile
& "C:\Cardoso Customer App\nssm\nssm.exe" restart CardosoCaddy

# Stop / start
& "C:\Cardoso Customer App\nssm\nssm.exe" stop CardosoCaddy
& "C:\Cardoso Customer App\nssm\nssm.exe" start CardosoCaddy
```

8.2 Logs

<code>C:\Caddy\logs\access.log</code>	per-request log, rolled at 100MB
<code>C:\Caddy\logs\service.log</code>	Caddy stdout
<code>C:\Caddy\logs\service-error.log</code>	Caddy stderr – parse errors, cert issues

The Cardoso Node logs stay where they always were:

```
C:\Cardoso Customer App\logs\service*.log
```

8.3 Certificate renewal — automatic

The Tailscale-issued cert is a standard Let's Encrypt cert with a **90-day lifetime**. Tailscale auto-renews about 30 days before expiry, as long as the machine stays on the tailnet. **No operator action is required.**

The Hub's **Settings** → **TLS** tab shows days-until-expiry — if you ever see that drop below 30, something's wrong with auto-renewal and you should investigate.

8.4 Certificate renewal — manual

If you ever need to force a renewal (e.g. after a hostname change or a cert file got corrupted), you have two options:

From the UI

Settings → **TLS** → **Renew cert now**. This re-runs `tailscale cert` and restarts Caddy automatically. It's audit-logged.

From the command line

```
cd C:\Caddy
tailscale cert <machine>.<tailnet>.ts.net
& "C:\Cardoso Customer App\nssm\nssm.exe" restart CardosoCaddy
```

8.5 Changing the Hub's hostname

If the Hub's tailnet name changes (renamed machine, moved tailnet):

1. Re-run `install-hub-caddy.ps1` with the new `-Hostname` .
2. The script issues a fresh cert, regenerates the Caddyfile, and reinstalls the service.
3. Update every site's `HUB_URL` to match.
4. Old `.crt` / `.key` files in `C:\Caddy\` can be deleted manually if you want to keep the directory tidy.

9. Recovery and troubleshooting

9.1 Caddy won't start

```
Get-Content C:\Caddy\logs\service-error.log -Tail 30
```

Common causes:

- **Port 443 already in use.** Something else (IIS, another reverse proxy) is bound to 443. Find it with `netstat -ano | findstr :443` and stop or reconfigure it.
- **Cert files missing.** Re-run the installer with `-SkipDownload` to re-issue the cert without re-downloading Caddy.
- **Caddyfile parse error.** Validate manually:

```
& "C:\Caddy\caddy.exe" validate --config "C:\Caddy\Caddyfile"
```

- **NSSM service is wedged.** Remove and reinstall:

```
cd "C:\Cardoso Customer App\scripts"
.\uninstall-hub-caddy.ps1
.\install-hub-caddy.ps1 -Hostname "<hostname>"
```

9.2 Cert expired or invalid

Run `tailscale cert <hostname>` manually to re-issue, then restart Caddy.

If `tailscale cert` itself fails:

- Verify `tailscale status` shows the machine as connected.
- Re-confirm HTTPS Certificates is **on** in the admin panel (someone may have toggled it off).
- Check `tailscale netcheck` for connectivity issues — the cert workflow needs the control plane to be reachable.

9.3 Sites can't reach the Hub

Symptoms: site logs show connection-refused, timeouts, or TLS errors when posting to the Hub.

Walk through:

1. **Did `BIND_ADDRESS=127.0.0.1` get set without Caddy being installed?** Look at the Hub's `.env`. If yes, either install Caddy or comment out that line and restart Node.
2. **Site `HUB_URL` still HTTP or wrong port?** Should be `https://<hostname>` (no port). Update the site's `.env` and restart.
3. **Tailscale connectivity?** From the site machine:

```
Test-NetConnection <hostname> -Port 443
```

`TcpTestSucceeded: True` means the network path is fine. False means tailnet routing is broken between site and Hub.

4. **Cert chain validation failure?** From the site, hit the URL with PowerShell to see the exact error:

```
Invoke-WebRequest -Uri "https://<hostname>/api/health"
```

A "could not establish trust relationship" error means the system trust store doesn't trust Let's Encrypt — extremely rare on Windows, indicates the site machine is missing recent root CA updates.

9.4 Full rollback to plain HTTP

If you need to revert (testing, or HTTPS introduces a regression you can't fix immediately):

```
cd "C:\Cardoso Customer App\scripts"  
.\uninstall-hub-caddy.ps1
```

Then edit `C:\Cardoso Customer App\.env` :

- Remove `BIND_ADDRESS=127.0.0.1` (or change to `0.0.0.0`)
- Remove `TLS_FRONTING=true`

Restart `CardosoCigarettes`. The Hub is back on plain HTTP at port 3001 on the LAN.

Sites need their `HUB_URL` reverted to `http://<hostname>:3001` until HTTPS is reinstated.

Appendix A: The generated Caddyfile

For reference, the `install-hub-caddy.ps1` script writes a Caddyfile that looks like this (with `<hostname>` and `<backend_port>` substituted):

```
{  
    # Skip Caddy's own certificate management – we use the cert  
    # files Tailscale wrote, not Caddy's auto-HTTPS.  
    auto_https disable_certs  
  
    # Logs go to the Caddy logs dir.  
    log {  
        output file C:/Caddy/logs/access.log {  
            roll_size 100mb  
            roll_keep 10  
            roll_keep_for 720h  
        }  
    }  
}
```

```

    }
    format json
  }
}

<hostname> {
  tls C:/Caddy/<hostname>.cert C:/Caddy/<hostname>.key
  encode gzip
  reverse_proxy http://127.0.0.1:<backend_port>
}

```

If you ever need to hand-edit this — to add a header, tune the proxy timeout, etc. — make the change, run `caddy validate --config "C:\Caddy\Caddyfile"`, then `nssm restart CardosoCaddy`.

Appendix B: Glossary

- **Tailnet** — a private network of machines authenticated to your Tailscale account. Your tailnet has a name like `tail-9a8b7.ts.net`.
- ***.ts.net** — the DNS suffix Tailscale issues to every machine in every tailnet. Resolvable only on the tailnet itself.
- **MagicDNS** — Tailscale's automatic DNS that resolves `<machine>.<tailnet>.ts.net` to the machine's tailnet IP.
- **DNS-01 challenge** — the Let's Encrypt challenge type that proves domain ownership via a DNS record rather than an HTTP request. The only challenge type that works for `*.ts.net` names because there's no public HTTP path to those machines.
- **NSSM** — "Non-Sucking Service Manager", a small wrapper that lets you run any executable (in our case `caddy.exe`) as a Windows service with log rotation, restart-on-failure, and clean stop signalling. Already bundled with the Cardoso Customer App installer.
- **Reverse proxy** — a server that accepts client connections, then forwards them to a backend server, optionally translating between protocols (e.g. HTTPS in, HTTP out). Caddy plays this role here.
- **HSTS** (Strict-Transport-Security) — a response header that tells browsers "for the next N seconds, only ever talk to this host over HTTPS, refuse plain HTTP". Enabled when `TLS_FRONTING=true`.
- **CSP** (Content-Security-Policy) — a response header that tells browsers which sources of script, style, image etc. are trusted. `upgrade-insecure-requests` is a CSP directive that auto-rewrites any `http://` URL on the page to `https://`. Enabled when `TLS_FRONTING=true`.

Appendix C: Files and where they live

C:\Caddy\	
├─ caddy.exe	the Caddy binary
├─ Caddyfile	generated by the install script
├─ <hostname>.cert	issued by tailscale cert
├─ <hostname>.key	issued by tailscale cert
└─ logs\	
├─ access.log	per-request log
├─ service.log	Caddy stdout
└─ service-error.log	Caddy stderr
C:\Cardoso Customer App\	

— nssm\nssm.exe	reused by the Caddy installer
— scripts\install-hub-caddy.ps1	the install script
— scripts\uninstall-hub-caddy.ps1	clean removal
— .env	BIND_ADDRESS, TLS_FRONTING set here
— logs\service*.log	Cardoso Node logs (unchanged)